Compilers

Guillermo A. Pérez

Lecture 2: Regular and context-free tools (abridged)

TL;DR: This lecture in short

What is a scanner? Why study them?

The scanner splits the input into sub-strings and groups them into lexical units. That is, it feeds a tokenized version of the input to the parse.

What is a grammar? How can they help with compiling?

A grammar is a way of specifying a formal language. We will use them to specify the full syntax of a programming language.

Required and target competences

What tools do we need?

Formal language theory (languages and machines, machines and computability);

What skills will we obtain?

- theory: regular languages, how to add some look-ahead, (context-free) grammars, pushdown automata
- practice: how to generate scanners automatically using flex, how to implement parsers

How will these skills be useful?

- It will allow you to build your compiler without hand-crafting a scanner or parser.

Scanning outline

1. Scanner building: manual vs. automatic
2. ~~Regular expressions~~
3. ~~Nondeterministic finite automata~~
4. ~~(Minimal) deterministic finite automata~~
5. Recognizing vs. scanning
6. Ensuring maximal matches
7. Dealing with output
8. A scanner-generating tool: flex

Building scanners

Before DFA-based scanners

Lexical analyzers used to be coded by hand before the advent of scanner-generating tools based on deterministic finite automata.

Advantages of automatically-generated ones

- The DFAs can be automatically constructed from regular expressions for the lexical units.
- DFA-based tools implicitly factor (potentially infinitely many) prefixes common to more than one lexeme! This has to be done manually otherwise

Scanner-building example

Example

Consider the extended regular expressions:

- `integer : [0-9]+`
- `real : [0-9]+"."[0-9]*|.[0-9]+`

Building a scanner for them would require factoring out the common `[0-9]+` prefix.

Hand-built scanners (1/2)

```
1  if (Character.isDigit(c)) {
2    // match an integer
3    if (c == '.') {
4      // match another integer
5      return new Token(REAL);
6    } else {
7      return new Token(INT);
8    }
9  } else if (c == '.') {
10   // match a float starting with .
11   return new Token(REAL);
12 } else ...
```

Hand-built scanners (2/2)

Advantages of hand-building them

- One can go beyond regular languages! (Most programming languages are Turing powerful.)
- Hand-written code is easier to debug than DFA simulators.
- Well-tuned hand-written scanners are more efficient.

In this course

We will use DFA-based scanner-generating tools such as flex and ANTLR. (Most such tools allow to recognize more than just regular languages...)

Example: syntax specification for SLPs

Straight-line programs

The language of SLPs consists of assignments of constants and simple arithmetic expressions:

```
LHS = RHS
LHS = RHS
...
```

- The left-hand side of the assignment is a simple identifier. That is, a letter followed by one or more letters or digits.
- The right-hand side is one of the following:
  - an integer (seq. of digits, with − maybe)
  - `ID + ID`, where `ID` is an identifier;
  - `ID - ID`; `ID * ID`.

Building expressions and automata

Exercise

Write regular expressions to recognize all lexical units in the SLP language.

Exercise

Draw/describe DFAs for the lexical units in the SLP language.

Recognizing is not the same as scanning

Recognizing lexical units using

Given a single string, we simulate the DFA and we get a Boolean answer: it is (not) a lexeme recognized by it.

We are in the scanning business

Given a "long" string,

- a scanner returns a sequence of tokens.
- Every token is a longest match, i.e. it is a maximal munch off the input string.
- For every such match, the scanner returns a token (not a lexeme!).

Munching like the best

Longest matches

We have to find the longest prefix of the remaining string which is in a lexical unit.

- We need some look-ahead to check if the currently matched string can be extended and still recognized by some DFA.
- However, we cannot get stuck while trying to extend the match...

Solution

The coder's solution: wrap the DFA simulation

Look-ahead corresponds to having access to more than one letter of the input at a time without consuming it. To solve the above problems scanner generators can

- keep simulating DFAs as long as at least one of them is not in its rejecting sink.
- One keeps track of the last accept position ℓ and state (per DFA) as well as the input from ℓ onward. We then simulate all DFAs until they reject or the input ends and output the last accept token.

One last problem: multiple matches?

If two DFAs recognize the same string...

The answer depends on the scanner generator you are using.

1. Some allow for multiple matches and prioritize the one corresponding to the first regular expression in the specification of the scanner.
2. Some completely disallow multiple matches, so the specifications must be unambiguous in that sense.

How to implement them?

1. Keep the DFAs ordered, and choose the first DFA which accepts.
2. The second option requires some language checking when generating the scanner... For all pairs of EREs with languages L_1, L_2, we should check: $L_1 \cap L_2 = \varnothing$.

Fast lexical analyzer generator

flex

Flex (short for fast lexical analyzer generator) is a program that generates scanners.

- Scanners are specified using regular expressions.
- Generated scanners split input strings into tokens via the use of DFAs.

Parsing outline

1. Grammars
2. Context-free languages
3. Derivation trees
4. The membership problem for CFGs
5. Pushdown automata
6. Grammar transformations

The limits of regular languages

Why do we not stick to regular expressions?

In specifying programming languages, we would like to make sure that expressions are, a.o., well-parenthesized.

- The language $L_{()}$ is the set of all words on $\Sigma = \{(,)\}$ such that every) matches a previous (and every (is eventually matched to a).
- $L_{()}$ is not regular!

A simple counter goes a long way

As it turns out, $L_{()}$ can be recognized by a DFA augmented with a counter. We will go even further and add a stack!

Grammars

Definition (Grammar)

A grammar is a tuple $G = (V, T, P, S)$ where

- V is a finite set of variables,
- T is a finite set of terminals,
- P is a finite set of production rules of the form $\alpha \to \beta$ with
  - $\alpha \in (V \cup T)^* V (V \cup T)^*$ and
  - $\beta \in (V \cup T)^*$,
- $S \in V$ is a variable called the start symbol.

Exercise

What is the grammar G_{SLP} for SLPs?

Example: a grammar with 3 counters

Consider $G_{ABC} = (\{A, B, C, S, S'\}, \{a, b, c\}, P, S)$ where P contains

(1)	S	$\to$	ε
(2)		$\to$	S'
(3)	S'	$\to$	$ABCS'$
(4)		$\to$	ABC
(5)	AB	$\to$	BA
(6)	BA	$\to$	AB
(7)	AC	$\to$	CA
(8)	CA	$\to$	AC
(9)	BC	$\to$	CB
(10)	CB	$\to$	BC
(11)	A	$\to$	a
(12)	B	$\to$	b
(13)	C	$\to$	c

Exercise

Question

Argue that *aabcbc* and *bcacab* are both in the language of the grammar. What is the language?

Formalizing what we just did

Definition (Derivation)

Let $\gamma \in (V \cup T)^* V (V \cup T)^*$ and $\delta \in (V \cup T)^*$. We say δ can be derived from γ iff there are $\gamma_1, \gamma_2 \in (V \cup T)^*$ and a rule $\alpha \to \beta \in P$ such that $\gamma = \gamma_1 \cdot \alpha \cdot \gamma_2$ and $\delta = \gamma_1 \cdot \beta \cdot \gamma_2$.

Definition (Sentential form)

A word $(V \cup T)^*$ is a sentential form if it can be derived from the start symbol.

Definition (Language of a grammar)

The language $L(G)$ of a grammar G is the set of its sentential forms w such that $w \in T^*$.

Types of grammars (1/2)

Do we want fully general grammars?

No. Most problems for grammars are undecidable! In particular, the halting problem for Turing machines reduces to the membership problem for grammars.

What is the problem with general grammars?

We want to generate parsers algorithmically from grammar specifications. The parser should be able to automatically detect whether a given sequence of tokens is part of the source programming language: the membership problem!

Types of grammars (2/2)

Definition (Grammar classes 0–3)

For all rules $\alpha \to \beta$

- (Class 0) All grammars: no restriction
- (Class 1) Context-sensitive grammars: either $\alpha = S$ and $\beta = \varepsilon$ or $|\alpha| \leq |\beta|$ and S does not appear in β
- (Class 2) Context-free grammars: $\alpha \in V$
- (Class 3) Regular grammars: $\alpha \in V$ and
  - left-regular: $\beta \in T^* \cup (V \cdot T^*)$
  - right-regular: $\beta \in T^* \cup (T^* \cdot V)$

Definition (Language classes)

A language L is Recursively enumerable, Context sensitive, Context free, or Regular, if there is a grammar G of the corresponding type s.t. $L(G) = L$.

A language hierarchy

Chomsky's hierarchy

The languages (not the grammars) form a strict hierarchy: $\mathrm{Reg} \subset \mathrm{CFL} \subset \mathrm{CSL} \subset \mathrm{RE}$.

Context-free languages

CFLs

A language L is context free if there exists a context-free grammar G, i.e. for all rules $\alpha \to \beta$ we have that α is a variable, such that $L(G) = L$.

Exercise

Is the following language context free?

$$L_{pal\#} = \{w \cdot \# \cdot w^R : w \in \{0,1\}^*\}$$

A simple grammar for palindromes

(1)	S	$\rightarrow$	$0S0$
(2)		$\rightarrow$	$1S1$
(3)		$\rightarrow$	$\#$

Does it facilitate implementing a recognizer?

Following the grammar's rules, we could

1. Read a 0;
2. then check that it is followed by a palindrome, i.e. a word that can be generated by S;
3. and finally read a matching 0.

Do the same thing if the first character we find is a 1.

A Python implementation

```
1   def S():
2       n = read_next_character()
3       if n == '#': return True
4       if n == '0':
5           r = S()
6           if (!r): return False
7           n = read_next_character()
8           if (n == '0'): return True
9           else: return False
10      if n == '1':
11          r = S()
12          if (!r): return False
13          n = read_next_character()
14          if (n == '1'): return True
15          else: return False
```

Questions

- Why can't this be encoded into a DFA?
- What unbounded information do we have to store?

Derivations for context-free grammars

Sequences of derivations

To test whether a word w is in the language of a CFG G, we derive sentential forms until we obtain w.

- Which variable should we choose to apply a rule and derive the next sentential form?

Definition (Left- and right-most derivations)

Respectively, always choose the left-most or right-most variable in the current sentential form.

Example: left-most and right-most derivations

(1)	Exp	$\rightarrow$	$\text{Exp} + \text{Exp}$
(2)		$\rightarrow$	$\text{Exp} * \text{Exp}$
(3)		$\rightarrow$	(Exp)
(4)		$\rightarrow$	Id
(5)		$\rightarrow$	Cst

Exercise

- Argue that the word $\text{Id} + \text{Id} * \text{Id}$ is in the language of the above grammar; and
- do so using left-most derivations or right-most derivations only.

Derivation trees

Definition (Derivation tree)

An ordered tree $\mathcal{T}$ is a derivation tree of $w \in \Sigma^*$ if and only if the root of $\mathcal{T}$ is S, w labels the leaves of $\mathcal{T}$ from left to right, the internal nodes of $\mathcal{T}$ have parent-children edges respecting the rules of the grammar:

- if n is an internal node of $\mathcal{T}$ with children $m_1, \ldots, m_k$ then there is a rule $\alpha \to \beta_1 \ldots \beta_k$ such that the sub-tree rooted at m_i is a derivation tree for the grammar with start symbol β_i.

Exercise

- Draw a derivation tree for the previous exercise.

Derivation ambiguity

How many derivation trees are there?

Unfortunately, there sometimes are several derivation trees for the same word and grammar.

Definition (Ambiguous grammar)

We say a CFG is ambiguous if there exists a word w in its language such that w has more than one derivation tree.

- Ambiguity cannot always be removed. (Some CFLs only have ambiguous grammars!)

Exercise

- Argue that the arithmetic-expression grammar from previous examples is ambiguous.

Things were nice with regular languages

Linear-time recognition

Given a regular expression E, we can build a DFA (in polynomial time). Then, for any given word w, in time $\mathcal{O}(|w|)$ we can determine if it is in $L(E)$ by simulating the DFA.

Definition (Membership problem for CFGs)

Given a CFG G and a word w, determine if $w \in L(G)$.

Efficiency

Is the above as efficient as it was for regular languages?

Chomsky normal form

Definition (CNF)

A CFG G is said to be in Chomsky normal form if all of its production rules are of the form

(1)	A	$\to$	BC
(2)	A	$\to$	a
(3)	S	$\to$	ε

where A, B, C are variables, a is a terminal, and B and C cannot be the start symbol S.

Anything can be put into CNF

Theorem (Universality of CNF)

Any CFG can be put into CNF by following the sequence of steps:

1. *Eliminate the start symbol from the right-hand sides.*
2. *Eliminate rules with non-solitary terminals.*
3. *Eliminate right-hand sides with more than 2 nonterminals.*
4. *Eliminate ε production rules.*
5. *Eliminate unit rules.*

Using CNF to decide membership

CNF "splits"

For grammars in CNF, A can generate w if we can find a rule $A \to BC$ and we can split w into two non-empty words $w = u \cdot v$ such that

- B generates u and
- C generates v.

The CYK algorithm

We build a table $\mathrm{Tab} : \{1, \ldots, |w|\}^2 \to 2^V$ as follows.

1. For all $1 \leq i \leq |w|$, we put variable A in $\mathrm{Tab}(i, i)$ iff the rule $A \to w_i$ occurs in the grammar.
2. For all $2 \leq \ell \leq |w|$, we put variable A in $\mathrm{Tab}(i, i + \ell)$ iff there is a rule $A \to BC$ in the grammar, and a split position $i \leq k < i + \ell$ s.t. $B \in \mathrm{Tab}(i, k)$ and $C \in \mathrm{Tab}(k + 1, i + \ell)$.

Good enough?

Algorithm analysis

Given a CFG G, we can get its CNF version G'. Then, for any given word w, the previous algorithm gives a way of deciding whether $w \in L(G') = L(G)$ in polynomial time. More precisely, in $\mathcal{O}(|w|^3)$.

Can we do better?

We will learn about deterministic pushdown-automata techniques which do better for some sub-classes of context-free languages.

Pushdown automata

Definition (Nondeterministic pushdown automata)

A nondeterministic pushdown automaton A is a tuple $(Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$ such that

- Q is a finite set of states;
- Σ is a finite input alphabet;
- Γ is a finite stack alphabet;
- $\delta : Q \times (\Sigma \cup \{\varepsilon\}) \times \Gamma \to 2^{Q \times \Gamma^*}$ is the transition function;
- $q_0 \in Q$ is the initial state;
- $Z_0 \in \Gamma$ is the initial symbol on the stack; and
- $F \subseteq Q$ is the set of accepting states.

Intuition

Informal description

From state q, it pops the top-of-stack symbol γ and reads an input letter σ, then transitions to a new state q' allowed by δ and pushes into the stack $\rho \in \Gamma^*$. All of this, if $\delta(q, \sigma, \gamma) \ni (q', \rho)$.

PDAs and CFGs

Theorem (CFLs)

Every CFG G can be transformed into an equivalent PDA A. The derivation process of the CFG is simulated by A in a left-most manner. Conversely, for every PDA, one can construct a CFG defining the same language.

Exercise

- Construct a PDA recognizing the language $L_{\mathrm{pal}\#}$.
- Construct a PDA recognizing the arithmetic-expression language.

Deterministic pushdown automata

Definition (Deterministic PDAs)

A PDA is deterministic if $\delta : Q \times (\Sigma \cup \{\varepsilon\}) \times \Gamma \to 2^{Q \times \Gamma^*}$ is such that:

- for all $q \in Q$, all $a \in \Sigma \cup \{\varepsilon\}$, and all $\gamma \in \Gamma$: $\delta(q, a, \gamma)$ has at most one element; and
- for all $q \in Q$ and all $\gamma \in \Gamma$: if $\delta(q, \varepsilon, \gamma) \neq \varnothing$ then $\delta(q, a, \gamma) = \varnothing$ for all $a \in \Sigma$.

Not CFL-universal

Unfortunately, not every context-free language can be recognized by a deterministic PDA. The CFL L_{pal}, for instance, cannot be recognized by a deterministic PDA.

Conclusions

- Grammars are too powerful for automatic recognition
- CFGs allow for efficient recognition
- CFGs and PDAs define the same class of languages, but we want a deterministic kind of machine
- Grammars can be easily (and automatically) simplified for later treatment by parsers